Name:- Raghav Anil Bhise
Roll No. :- 391061
PRN:- 22311792

1. Title

Implementation of Text Preprocessing Techniques in Natural Language Processing

---

2. Aim

To study and implement various **text preprocessing techniques** to convert raw textual data into a clean and structured form suitable for Natural Language Processing tasks.

---

3. Objectives

- To understand the role of text preprocessing in NLP
- To implement basic preprocessing steps such as lowercasing, tokenization, stopword removal, stemming, and lemmatization
- To transform raw text into a normalized and machine-processable format
- To prepare text data for further NLP operations like feature extraction and modeling

---

4. Theory

4.1 Text Preprocessing in NLP

Text preprocessing is a crucial step in Natural Language Processing that involves cleaning and normalizing raw text data. Real-world text is often noisy, unstructured, and inconsistent, which can negatively affect NLP models.

4.2 Common Text Preprocessing Steps (As per the File)

a) Lowercasing

Converts all text into lowercase to ensure uniformity and avoid duplicate word representations.

b) Tokenization

Splits text into smaller units called **tokens**, usually words or sentences.

c) Stopword Removal

Removes commonly used words (such as *is, the, and*) that do not add significant meaning to the text.

d) Stemming

Reduces words to their root form by removing suffixes.

e) Lemmatization

Converts words into their meaningful base form using grammatical rules and vocabulary.

4.3 Importance of Text Preprocessing

- Reduces noise in textual data
- Improves accuracy and efficiency of NLP models
- Helps in better feature extraction
- Acts as a foundation for advanced NLP techniques

---

5. Program Codes

Removal of stop words

[1]

```
#install NLTK library

!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
```

[3]

```
sentence1 = "The quality of the product is excellent"

sentence2 = "The packaging is attractive"

sentence3 = "The product meets basic expectations"
```

[4]

```
from nltk.corpus import stopwords
```

[5]

```
import nltk
nltk.download('stopwords')
stop_words = stopwords.words('english')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

[6]

```
stop_words
```

```
['a',
 'about',
 'above',
 'after',
 'again',
 'against',
 'ain',
 'all',
 'am',
 'an',
 'and',
 'any',
 'are',
 'aren',
 "aren't",
 'as',
 'at',
 'be',
 'because',
 'been',
 'before',
 'being',
 'below',
 'between',
 'both',
 'but',
 'by',
 'can',
 'couldn',
 "couldn't",
 'd',
 'did',
 'didn',
 "didn't",
```

```
 'yourself',
 'yourselves',
 "you've"]
```

```
[7]
len(stop_words)
```

```
198
```

```
[8]
stop_words.append('I')
```

```
[9]
stop_words
```

```
 'their',
 'theirs',
 'them',
 'themselves',
 'then',
 'there',
 'these',
 'they',
 "they'd",
 "they'll",
 "they're",
 "they've",
 'this',
 'those',
 'through',
 'to',
 'too',
 'under',
 'until',
 'up',
 've',
 'very',
 'was',
```

```
[10]
def remove_stopwords(text):

  #lower text
  text = text.lower()

  #split text
  text = text.split()

  return text

remove_stopwords(sentence1)
```

```
['the', 'quality', 'of', 'the', 'product', 'is', 'excellent']
```

```
[11]
def remove_stopwords(text):

  #lower text
  text = text.lower()

  #split text
  text = text.split()

  for word in text:
    if word in stop_words:
      text.remove(word)

  return text

remove_stopwords(sentence3)
```

```
['product', 'meets', 'basic', 'expectations']
```

---

6. Result and Outcomes

Result

- The raw input text was successfully preprocessed.
- Text was converted to lowercase, tokenized, and cleaned.
- Stopwords were removed, and words were normalized using stemming and lemmatization.

Outcomes

- Understood the complete text preprocessing pipeline
- Learned how different preprocessing steps improve text quality
- Gained hands-on experience using NLTK for NLP preprocessing
- Prepared text data for further NLP tasks such as vectorization and classification

---

7. Conclusion

Text preprocessing is a vital step in Natural Language Processing that ensures raw text is clean, consistent, and suitable for computational analysis. By implementing multiple preprocessing techniques, this experiment demonstrated how noise can be reduced and meaningful information can be extracted effectively, forming a strong base for advanced NLP applications.